

Testkonzept für das Open-Source-Projekt „lazygit„

Belegarbeit im Studiengang Informatik (B.Sc.)

**Grundlagen des Softwaretestens
Wintersemester 2025/26**

Dozent: Prof. Dr. Matthias Längrich

Hochschule Zittau/Görlitz
Fakultät Elektrotechnik und Informatik

**Verfasser: Konrad Miosga
Matrikelnummer: 1056710
E-Mail-Adresse: konrad.miosga@stud.hszg.de**

Datum der Abgabe: 22.01.2026

Inhaltsverzeichnis

1. Einleitung	1
1.1. Projektkontext	1
1.2. Zielsetzung und Methodik	1
2. Theoretische Grundlagen	2
3. Analyse der vorhandenen Teststrategie	4
3.1. Unit-Tests	4
3.2. Table-Driven Tests	4
3.3. Integrationstests	5
4. Continuous Integration Pipeline	8
5. Entwurf eigener Testfälle	9
5.1. Identifikation von Testlücken	9
5.2. Analyse und Testfalldesign - tag.go	10
5.2.1. Anforderungsanalyse	10
5.2.2. Use-Case-Analyse	10
5.2.3. Testbedingungen	10
5.2.4. Testfalldesign	11
5.3. Implementierung - tag.go	12
5.4. Testergebnisse und Coverage-Verbesserung - tag.go	13
5.5. Analyse und Testfalldesign - string_stack.go	13
5.6. Implementierung - string_stack.go	13
5.7. Testergebnisse und Coverage-Verbesserung - string_stack.go	14
6. Testauswertung und Metriken	14
7. Fazit	14
Quellen	15
8. Anhang	15

1. Einleitung

1.1. Projektkontext

In dieser Belegarbeit wird das Testkonzept des Open-Source-Projekts **lazygit** analysiert, bewertet und durch eigene Testfälle ergänzt. Lazygit ist eine in Go entwickelte Terminal-UI für Git-Kommandos, die komplexe Git-Operationen durch eine intuitive, tastaturgesteuerte Benutzeroberfläche vereinfacht. Das Projekt wurde von Jesse Duffield initiiert, wird seit 2018 aktiv entwickelt und gehört mit über 50.000 GitHub-Stars zu den populärsten Git-Tools im Terminal-Bereich.

Die Architektur folgt dem MVC-Muster mit klarer Trennung zwischen UI-Komponenten, Business-Logik und Git-Operationen. Diese Struktur erleichtert das isolierte Testen einzelner Komponenten erheblich.

1.2. Zielsetzung und Methodik

Diese Arbeit verfolgt mehrere Ziele: Analyse der bestehenden Teststrategie mit ihren Stärken und Schwächen, Identifikation von Testlücken durch Coverage-Analysen und Code-Reviews, praktische Implementierung neuer Testfälle sowie Ableitung konkreter Handlungsempfehlungen.

Die Bearbeitung erfolgte systematisch: Nach Literaturrecherche zu Testing-Grundlagen wurde die bestehende Testinfrastruktur analysiert. Coverage-Reports und manuelle Code-Reviews identifizierten Testlücken, die anschließend durch table-driven Tests geschlossen wurden. Die neuen Tests wurden in die CI-Pipeline integriert und auf mehreren Plattformen validiert.

2. Theoretische Grundlagen

Software-Testing ist ein zentraler Bestandteil der Qualitätssicherung in der Softwareentwicklung. Das oft zitierte Statement von Edsger W. Dijkstra bringt dabei eine grundlegende Eigenschaft des Testens prägnant auf den Punkt: Tests eignen sich hervorragend zum Aufdecken von Fehlern, sind jedoch ungeeignet, um die vollständige Fehlerfreiheit eines Programms nachzuweisen [1].

Diese Einschränkung ergibt sich aus der Natur des Software-Testings als Stichprobenverfahren. Da Programme in der Regel eine sehr große Menge möglicher Eingaben besitzen, kann im Rahmen von Tests nur eine endliche Teilmenge davon überprüft werden. Ein formaler Beweis der Korrektheit allein durch Tests ist daher prinzipiell nicht möglich.

Vor diesem Hintergrund kommt der systematischen und zielgerichteten Auswahl repräsentativer Testfälle eine zentrale Bedeutung zu. Ziel des Testens ist es nicht, wie schon eingangs erwähnt, Fehlerfreiheit zu garantieren, sondern mit begrenztem Aufwand eine möglichst hohe Wahrscheinlichkeit zur Entdeckung relevanter Defekte zu erreichen.

Der Testprozess lässt sich grundsätzlich in zwei zentrale Bereiche gliedern: die statische Analyse und das dynamische Testen [2, S. 4]. Die statische Analyse untersucht Softwareartefakte ohne deren Ausführung und zielt darauf ab, potenzielle Fehler, Regelverletzungen oder Qualitätsmängel frühzeitig im Entwicklungsprozess zu identifizieren [2, S. 43–45]. Zu den typischen Verfahren zählen Code-Reviews, der Einsatz von Lintern sowie statische Datenfluss- und Kontrollflussanalysen.

Demgegenüber steht das dynamische Testen, bei dem die Software mit konkreten Eingabedaten ausgeführt wird. Dadurch lassen sich insbesondere solche Fehler aufdecken, die erst zur Laufzeit auftreten, etwa Speicherlecks, Race Conditions oder fehlerhaftes Laufzeitverhalten. Dynamische Tests ermöglichen somit eine Überprüfung des tatsächlichen Systemverhaltens unter realistischen oder gezielt konstruierten Bedingungen [2, S. 39–43]. Der Schwerpunkt dieser Arbeit liegt auf dem dynamischen Testen und den zugehörigen Testverfahren.

Dynamisches Testen lässt sich grundsätzlich in Black-Box-Tests und White-Box-Tests unterteilen. Black-Box-Tests leiten Testfälle ausschließlich aus Spezifikationen und Anforderungen ab, ohne Kenntnis der internen Programmstruktur. Typische Verfahren sind hierbei die Äquivalenzklassenbildung sowie darauf aufbauend die Grenzwertanalyse. White-Box-Tests hingegen konstruieren Testfälle auf Basis der internen Programmstruktur mit dem Ziel, definierte Abdeckungskriterien wie Anweisungs- oder Pfadabdeckung zu erfüllen [3, S. 173–174]. In der Testpraxis werden beide Ansätze häufig kombiniert, um sowohl funktionale als auch strukturelle Aspekte der Software zu überprüfen.

Innerhalb der White-Box-Tests lassen sich verschiedene, aufeinander aufbauende Testverfahren unterscheiden. Unit-Tests überprüfen die kleinsten testbaren Einheiten eines Programms in Isolation. Abhängige Komponenten werden dabei typischerweise durch Mocks oder Stubs ersetzt, was schnelle, reproduzierbare und deterministische Tests ermöglicht [2, S. 371–372]. Eine häufig eingesetzte Ausprägung sind sogenannte Table-Driven Tests, bei denen mehrere Testfälle in Form von Datentabellen definiert und von einem generischen Testcode iterativ ausgeführt werden [4]. Dieses Vorgehen reduziert Redundanz und verbessert die Wartbarkeit der Tests.

Aufbauend auf den Unit-Tests folgen Integrationstests, die das Zusammenwirken mehrerer bereits getesteter Module überprüfen. Ziel ist es insbesondere, Fehler an Schnittstellen sowie Probleme im Zusammenspiel der Komponenten aufzudecken [2, S. 372–376]. Im Gegensatz zu Unit-Tests kommen hierbei überwiegend reale Komponenten statt isolierender Mocks zum Einsatz, wodurch eine höhere Aussagekraft hinsichtlich der Gesamtfunktionalität des Systems erreicht wird.

3. Analyse der vorhandenen Teststrategie

Lazygit testet auf zwei Ebenen: Unit-Tests und Integrationstests. Die Unit-Tests prüfen einzelne Funktionen isoliert und laufen sehr schnell, während die Integrationstests die gesamte Anwendung mit echter Benutzerinteraktion testen. Dabei kommen sogenannte Table-Driven Tests zum Einsatz, eine Go-typische Methode, um viele ähnliche Testfälle kompakt zu schreiben.

3.1. Unit-Tests

Die Unit-Tests in Lazygit arbeiten mit Mocks, das heißt sie führen keine echten Git-Befehle aus, sondern simulieren diese. Das Kernstück ist der `FakeCmdObjRunner`, der vorgibt, Git-Befehle auszuführen, tatsächlich aber nur aufzeichnet, welche Befehle aufgerufen wurden, und vordefinierte Antworten zurückgibt. So kann man Tests schreiben, die schnell, zuverlässig und unabhängig vom tatsächlichen Git-Zustand sind.

Ein typischer Unit-Test sieht ungefähr so aus:

```
1 func TestBranchNewBranch(t *testing.T) {
2     runner := oscommands.NewFakeRunner(t).
3     ExpectGitArgs([]string{"checkout", "-b", "test", "refs/heads/master"}, "", nil)
4     instance := buildBranchCommands(commonDeps{runner: runner})
5
6     assert.NoError(t, instance.New("test", "refs/heads/master"))
7     runner.CheckForMissingCalls()
8 }
```

Man sagt dem Fake-Runner vorher, welchen Git-Befehl man erwartet, führt dann die zu testende Funktion aus, und prüft am Ende, ob wirklich der erwartete Befehl aufgerufen wurde. Das ist einfach und funktioniert gut für Komponenten, die hauptsächlich Git-Befehle zusammenbauen und ausführen.

3.2. Table-Driven Tests

Table-Driven Tests sind in Go sehr verbreitet. Die Idee ist, dass man mehrere Testfälle in einer Liste definiert und dann in einer Schleife durchläuft. Das spart Code-Duplikation und macht es einfach, neue Testfälle hinzuzufügen. In Lazygit wird das konsequent eingesetzt, besonders wenn man verschiedene Eingaben und Fehlerfälle durchprobieren will.

Ein Beispiel aus `branch_test.go` zeigt, wie das aussieht:

```
1  scenarios := []scenario{ go
2      {
3          "Can't retrieve pushable count",
4          oscommands.NewFakeRunner(t).
5              ExpectGitArgs([]string{"rev-list", "@{u}..HEAD", "--count"}, "",
6              errors.New("error")),
7          "?", "?",
8      },
9      {
10         "Retrieve pullable and pushable count",
11         oscommands.NewFakeRunner(t).
12             ExpectGitArgs([]string{"rev-list", "@{u}..HEAD", "--count"}, "1\n", nil).
13             ExpectGitArgs([]string{"rev-list", "HEAD..@{u}", "--count"}, "2\n", nil),
14         "1", "2",
15     },
16 }
17 for _, s := range scenarios {
18     t.Run(s.testName, func(t *testing.T) {
19         instance := buildBranchCommands(commonDeps{runner: s.runner})
20         pushables, pullables := instance.GetCommitDifferences("HEAD", "@{u}")
21         assert.EqualValues(t, s.expectedPushables, pushables)
22         assert.EqualValues(t, s.expectedPullables, pullables)
23         s.runner.CheckForMissingCalls()
24     })
25 }
```

Jedes Szenario hat einen Namen, eine Mock-Konfiguration und erwartete Ergebnisse. Die Schleife führt dann für jeden Fall den gleichen Test aus. Wenn ein Test fehlschlägt, sieht man sofort am Namen, welches Szenario das Problem hat.

3.3. Integrationstests

Die Integrationstests sind das Herzstück der Teststrategie. Sie testen nicht einzelne Funktionen, sondern komplette User-Workflows. Dafür wurde ein eigenes Test-Framework entwickelt, mit der man UI-Interaktionen beschreiben kann.

Ein Test für einen Rebase sieht zum Beispiel so aus:

```
1  var Rebase = NewIntegrationTest(NewIntegrationTestArgs{ go
2      Description: "Rebase onto another branch, deal with the conflicts.",
3      SetupRepo: func(shell *Shell) {
4          shared.MergeConflictsSetup(shell)
5      },
```

```

6      Run: func(t *TestDriver, keys config.KeybindingConfig) {
7          t.Views().Commits().TopLines(
8              Contains("first change"),
9              Contains("original"),
10         )
11
12         t.Views().Branches().
13             Focus().
14             Lines(
15                 Contains("first-change-branch"),
16                 Contains("second-change-branch"),
17             ).
18             SelectNextItem().
19             Press(keys.Branches.RebaseBranch)
20
21         t.ExpectPopup().Menu().
22             Title(Equals("Rebase 'first-change-branch'")).
23             Select(Contains("Simple rebase")).
24             Confirm()
25
26         t.Common().AcknowledgeConflicts()
27     },
28 })

```

Der Test läuft in drei Schritten ab: Zuerst wird ein echtes Git-Repository mit dem Shell-Helper vorbereitet. Dann simuliert der TestDriver Benutzeraktionen wie `Focus()`, `SelectNextItem()` oder `Press()` – das entspricht den Tastendrücken, die ein echter Nutzer machen würde. Zwischendurch wird immer wieder geprüft, ob die UI das Erwartete anzeigt, zum Beispiel mit `Contains()` oder `Equals()`. Man kann den Test lesen wie eine Beschreibung dessen, was ein Nutzer tut.

Es gibt über 450 solcher Integrationstests, die alle möglichen Szenarien abdecken: Branch-Operationen wie Checkout oder Rebase, Commit-Operationen wie Amend oder Cherry-Pick, interaktive Rebases, Konfliktbehandlung, File-Operations und mehr.

Technisch basiert das Framework auf einer Architektur spezialisierter Driver-Komponenten. Der ViewDriver ermöglicht Interaktionen mit Listen-Views wie Branches, Commits und Files, während der MenuDriver die Navigation in Popup-Menüs steuert. Der PromptDriver behandelt Texteingaben, der ConfirmationDriver das Bestätigen oder Ablehnen von Dialogen und der AlertDriver die Validierung von Fehlermeldungen. Diese Abstraktion entkoppelt die Testlogik von der konkreten UI-Implementation, was Refactorings erheblich erleichtert. Ändert sich die

Struktur der Benutzeroberfläche, müssen lediglich die Driver-Implementierungen angepasst werden, während die hunderten von Tests unverändert bleiben können.

4. Continuous Integration Pipeline

Die CI-Pipeline ist kein Bestandteil der Teststrategie selbst, sondern dient als technisches Mittel zur automatisierten Umsetzung der beschriebenen Testkonzepte. Lazygit nutzt GitHub Actions, um bei jedem Push und Pull Request automatisch die Qualitätssicherung durchzuführen.

Die Pipeline führt parallel mehrere Prüfungen aus: Unit-Tests laufen auf Ubuntu und Windows (Cross-Platform-Kompatibilität), Integrationstests validieren die Funktionalität gegen vier verschiedene Git-Versionen (2.32.0 bis neueste), Build-Jobs kompilieren für Linux, Windows und macOS, Konsistenz-Checks prüfen Dependencies und generierte Dateien, und golangci-lint führt statische Code-Analyse durch. Nach erfolgreicher Testausführung werden Coverage-Daten zu Codacy hochgeladen.

5. Entwurf eigener Testfälle

Der praktische Teil dieser Arbeit bestand darin, Testlücken im Lazygit-Projekt zu identifizieren und durch eigene Testfälle zu schließen. Dabei wurden die zuvor analysierten Teststrategien angewendet und die Coverage messbar verbessert.

5.1. Identifikation von Testlücken

Die Analyse der Coverage-Reports in Kombination mit manuellen Code-Reviews identifizierte zwei signifikante Testlücken. Die Tag-Kommandos in `pkg/commands/git_commands/tag.go` enthielten acht Funktionen ohne jegliche Testabdeckung. Tags sind in Git zentral für die Versionsverwaltung, weshalb fehlerhafte Implementierungen zu versehentlich überschriebenen oder falsch gesetzten Tags führen können.

1	<code>/lazygit/pkg/commands/git_commands/tag.go:14:</code>	NewTagCommands	0.0%	bash
2	<code>/lazygit/pkg/commands/git_commands/tag.go:20:</code>	CreateLightweightObj	0.0%	
3	<code>/lazygit/pkg/commands/git_commands/tag.go:30:</code>	CreateAnnotatedObj	0.0%	
4	<code>/lazygit/pkg/commands/git_commands/tag.go:40:</code>	HasTag	0.0%	
5	<code>/lazygit/pkg/commands/git_commands/tag.go:49:</code>	LocalDelete	0.0%	
6	<code>/lazygit/pkg/commands/git_commands/tag.go:56:</code>	Push	0.0%	
7	<code>/lazygit/pkg/commands/git_commands/tag.go:71:</code>	ShowAnnotationInfo	0.0%	
8	<code>/lazygit/pkg/commands/git_commands/tag.go:80:</code>	IsTagAnnotated	0.0%	

Die zweite Lücke betraf die `StringStack`-Datenstruktur in `pkg/utils/string_stack.go`, eine LIFO-Implementierung ohne Tests.

1	<code>/lazygit/pkg/utils/string_stack.go:7:</code>	Push	0.0%	bash
2	<code>/lazygit/pkg/utils/string_stack.go:11:</code>	Pop	0.0%	
3	<code>/lazygit/pkg/utils/string_stack.go:21:</code>	IsEmpty	0.0%	
4	<code>/lazygit/pkg/utils/string_stack.go:25:</code>	Clear	0.0%	

Die Priorisierung erfolgte nach Kritikalität, wobei die Tag-Funktionen als wichtiger eingestuft wurden.

5.2. Analyse und Testfalldesign - tag.go

5.2.1. Anforderungsanalyse

Die Tag-Verwaltung in lazygit erfüllt zentrale Anforderungen der Git-Versionskontrolle:

Funktionale Anforderungen:

FA-01	Erstellung von Lightweight Tags (einfache Commit-Pointer)
FA-02	Erstellung von Annotated Tags mit Metadaten (Autor, Datum, Message)
FA-03	Tags auf beliebigen Commits erstellen (nicht nur HEAD)
FA-04	Überschreiben existierender Tags mit Force-Flag
FA-05	Lokales Löschen von Tags
FA-06	Unterscheidung zwischen Annotated und Lightweight Tags

5.2.2. Use-Case-Analyse

Aus der Anforderungsanalyse wurden vier zentrale Use Cases abgeleitet, die das Testdesign maßgeblich beeinflussen:

UC	Beschreibung	Priorität
UC-01	Release-Version taggen: Entwickler markiert Commits mit einer Versionen	Hoch
UC-02	Fehlerhaften Tag lokal korrigieren: Vor Remote-Push Fehler beheben	Mittel
UC-03	Tag-Informationen anzeigen: Unterscheidung Lightweight/Annotated	Hoch
UC-04	Bestehenden Tag verschieben: Rolling-Tags (latest, stable) aktualisieren	Mittel

5.2.3. Testbedingungen

Aus den Use Cases wurden sieben konkrete Testbedingungen abgeleitet:

ID	Bedingung	Erwartetes Verhalten
TB-01	Lightweight Tag ohne Ref	<code>git tag -- <name></code>
TB-02	Tag auf spezifischem Commit	<code>git tag -- <name> <ref></code>
TB-03	Force-Flag gesetzt	<code>--force</code> Flag inkludiert
TB-04	Force + spezifischer Commit	Kombination beider Flags
TB-05	Annotated Tag mit Message	<code>-m <message></code> Parameter
TB-06	Tag-Typ erkennen	<code>git cat-file -t</code> Output parsen
TB-07	Tag löschen	<code>git tag -d <name></code> ausführen

5.2.4. Testfalldesign

Die Tag-Funktionen konstruieren Git-Befehle programmatisch mit Hilfsfunktionen wie `NewGitCmd` und `ArgIf`, die Argumente nur unter bestimmten Bedingungen hinzufügen. Beispielsweise fügt `ArgIf(force, "--force")` das Force-Flag nur bei Bedarf hinzu. Diese bedingte Logik ist fehleranfällig und erfordert Tests für verschiedene Parameter-Kombinationen.

Entwurfstechniken:

Äquivalenzklassenbildung wurde für Parameter-Kategorisierung eingesetzt. Der Parameter `tagName` wurde in gültige semantische Versionen („v1.0.0“) und einfache Strings („latest“, „stable“) unterteilt. Der Parameter `ref` unterscheidet zwischen leerem String (HEAD), Commit-Hash und Branch-Namen. Diese Klassifizierung reduziert die Testanzahl bei gleichbleibender Abdeckung.

Kombinatorisches Testing (Pairwise) optimiert die Parameterkombinationen. Bei drei Parametern (`tagName`, `ref`, `force`) mit jeweils 2 Werten ergeben sich theoretisch 8 Kombinationen. Pairwise-Coverage reduziert dies auf 4 repräsentative Testfälle, die alle Paarkombinationen abdecken.

Grenzwertanalyse fokussiert Edge Cases bei der Git-Output-Verarbeitung. Die Funktion `IsTagAnnotated` muss mit verschiedenen Formaten umgehen: exakte Strings („tag“, „commit“), Whitespace-Varianten („ tag n“) und leere Ausgaben.

Für die Tag-Kommandos wurde Table-Driven Testing gewählt, da dies dem Projekt-Standard entspricht und sich optimal für Parametervariationen eignet. Die Tests für `CreateLightweightObj` decken vier Szenarien ab: einfacher Tag auf HEAD, Tag auf spezifischem Commit, Force-Flag zum Überschreiben existierender Tags und die Kombination aller Parameter. Für `CreateAnnotatedObj` wurden analoge Tests mit zusätzlichem Message-Parameter entwickelt. Die Funktion `IsTagAnnotated` parst Git-Ausgaben („commit“ vs. „tag“) und erhielt Tests mit verschiedenen Ausgabeformaten inklusive Whitespace-Varianten.

Die resultierende Test-Suite umfasst 12 Testfälle:

- 4 für `CreateLightweightObj` (TF-01 bis TF-04)
- 4 für `CreateAnnotatedObj` (TF-05 bis TF-08)

- 3 für IsTagAnnotated (TF-09 bis TF-11)
- 1 für LocalDelete (TF-12)

Diese Testfälle validieren alle sieben Testbedingungen und decken somit alle funktionalen Anforderungen ab.

5.3. Implementierung - tag.go

Die Tag-Tests wurden in tag_test.go implementiert und folgen strikt den Projekt-Konventionen. Jeder Testfall definiert eine Szenario-Struktur mit Eingabeparametern und erwarteten Git-Argumenten. Beispielhaft sei hier das implementierte Szenario für TF-01 dargestellt.

```

1  //...
2  func TestTagCommands_CreateLightweightObj(t *testing.T) {
3      type scenario struct {
4          testName      string
5          tagName        string
6          ref            string
7          force          bool
8          expectedCmdArgs []string
9      }
10
11     scenarios := []scenario{
12         {
13             testName:      "create simple lightweight tag on HEAD",
14             tagName:        "v1.0.0",
15             ref:            "",
16             force:          false,
17             expectedCmdArgs: []string{"git", "tag", "--", "v1.0.0"},
18         }
19     },
20     //...
21     for _, s := range scenarios {
22         t.Run(s.testName, func(t *testing.T) {
23             runner := oscommands.NewFakeRunner(t)
24             gitCommon := buildGitCommon(commonDeps{runner: runner})
25             tagCommands := NewTagCommands(gitCommon)
26
27             cmdObj := tagCommands.CreateLightweightObj(s.tagName, s.ref, s.force)
28
29             assert.Equal(t, s.expectedCmdArgs, cmdObj.Args())
30         })
31     }
32 }

```

Der FakeCmdObjRunner simuliert Git-Befehle ohne tatsächliche Ausführung. Für jeden Testfall werden die erwarteten Argumente beim Fake-Runner registriert, die Funktion ausgeführt und anschließend mit `CheckForMissingCalls()` validiert, dass alle erwarteten Befehle aufgerufen wurden.

Die vollständige Implementierung ist in [Abbildung 1](#) ersichtlich.

5.4. Testergebnisse und Coverage-Verbesserung - tag.go

Die Coverage-Analyse zeigt signifikante Verbesserungen für `tag.go`, wo fünf von acht Funktionen auf 100% Coverage gebracht wurden:

1	/lazygit/pkg/commands/git_commands/tag.go:14:	NewTagCommands	100.0%	bash
2	/lazygit/pkg/commands/git_commands/tag.go:20:	CreateLightweightObj	100.0%	
3	/lazygit/pkg/commands/git_commands/tag.go:30:	CreateAnnotatedObj	100.0%	
4	/lazygit/pkg/commands/git_commands/tag.go:40:	HasTag	0.0%	
5	/lazygit/pkg/commands/git_commands/tag.go:49:	LocalDelete	100.0%	
6	/lazygit/pkg/commands/git_commands/tag.go:56:	Push	0.0%	
7	/lazygit/pkg/commands/git_commands/tag.go:71:	ShowAnnotationInfo	0.0%	
8	/lazygit/pkg/commands/git_commands/tag.go:80:	IsTagAnnotated	100.0%	

Die drei verbleibenden Funktionen (`HasTag`, `Push`, `ShowAnnotationInfo`) sind Remote-Operationen, die Netzwerk-Kommunikation erfordern und besser durch Integrationstests abgedeckt werden. Die Gesamt-Coverage von `tag.go` stieg von 0% auf 62.5%.

5.5. Analyse und Testfalldesign - string_stack.go

Für `StringStack` wurde ein zustandsbasierter Testansatz gewählt. Die Tests validieren LIFO-Semantik (`TestStringStack_PushAndPop`), das Verhalten bei leerem Stack (`TestStringStack_PopEmptyStack`), Zustandsprüfung (`TestStringStack_IsEmpty`), vollständiges Zurücksetzen (`TestStringStack_Clear`) und komplexe Operationssequenzen (`TestStringStack_MultipleOperations`).

5.6. Implementierung - string_stack.go

Die `StringStack`-Tests verwenden klassisches Unit-Testing ohne Table-Driven-Ansatz, da primär Zustandsübergänge getestet werden:

```
1 func TestStringStack_PushAndPop(t *testing.T) {  
2     stack := NewStringStack()  
}
```

```

3     stack.Push("first")
4     stack.Push("second")
5
6     assert.Equal(t, "second", stack.Pop())
7     assert.Equal(t, "first", stack.Pop())
8 }

```

5.7. Testergebnisse und Coverage-Verbesserung - string_stack.go

1	/lazygit/pkg/utils/string_stack.go:7:	Push	100.0%	bash
2	/lazygit/pkg/utils/string_stack.go:11:	Pop	100.0%	
3	/lazygit/pkg/utils/string_stack.go:21:	IsEmpty	100.0%	
4	/lazygit/pkg/utils/string_stack.go:25:	Clear	100.0%	

Auf Package-Ebene verbesserte sich pkg/commands/git_commands von 37.1% auf 37.6% (+0.5 Prozentpunkte) und pkg/utils von 58.2% auf 59.6% (+1.4 Prozentpunkte). Obwohl die prozentualen Verbesserungen moderat erscheinen, schließen sie konkrete Lücken in wichtigen Funktionen innerhalb umfangreicher Packages.

6. Testauswertung und Metriken

Die Coverage-Verbesserungen sind messbar und signifikant. Für tag.go stieg die Coverage von 0% auf 62.5%, wobei alle getesteten Funktionen 100% Coverage erreichten. Nur drei Remote-Funktionen blieben ungetestet. string_stack.go erreichte vollständige 100% Coverage für alle Funktionen.

Auf Package-Ebene verbesserte sich pkg/commands/git_commands um 0.5 Prozentpunkte (37.1% → 37.6%) und pkg/utils um 1.4 Prozentpunkte (58.2% → 59.6%). Diese scheinbar kleinen Zahlen sind bedeutsam, da beide Packages umfangreich sind und die neuen Tests gezielt Lücken schließen.

Lazygit verfügt über 80+ Test-Dateien mit 2500 Unit-Test-Funktionen. Unit-Tests laufen in unter einer Minute. Die Test-Code-Ratio ist ausgewogen – kritische Packages haben umfangreichere Tests. Die Tests integrierten sich nahtlos in die CI-Pipeline durch Standard-Go-Patterns und laufen auf allen Plattformen.

7. Fazit

Quellen

- [1] E. W. Dijkstra, „The Humble Programmer“, *ACM Turing Award Lectures*. Association for Computing Machinery, New York, NY, USA, 2007. doi: 10.1145/1283920.1283927.
- [2] P. Liggesmeyer, *Software-Qualität: Testen, Analysieren und Verifizieren von Software*, 2. Auflage. Spektrum Akademischer Verlag, 2009.
- [3] D. W. Hoffmann, *Software-Qualität*, 2. Auflage. Springer Vieweg, 2013.
- [4] The Go Authors, „Table Driven Tests“. Zugriffen: 15. Januar 2026. [Online]. Verfügbar unter: <https://github.com/golang/go/wiki/TableDrivenTests>

8. Anhang

```
1 //tag_test.go
2 package git_commands
3
4 import (
5     "testing"
6
7     "github.com/jesseduffield/lazygit/pkg/commands/oscommands"
8     "github.com/stretchr/testify/assert"
9 )
10
11 func TestTagCommands_CreateLightweightObj(t *testing.T) {
12     type scenario struct {
13         testName      string
14         tagName        string
15         ref            string
16         force         bool
17         expectedCmdArgs []string
18     }
19
20     scenarios := []scenario{
21         {
22             //TF-01
23             testName:      "create simple lightweight tag on HEAD",
24             tagName:         "v1.0.0",
25             ref:             "",
26             force:           false,
27             expectedCmdArgs: []string{"git", "tag", "--", "v1.0.0"},
28         },
29         {
30             //TF-02
31             testName:      "create lightweight tag on specific commit",
32             tagName:         "v1.0.0",
```

```

33     ref:          "abc123",
34     force:        false,
35     expectedCmdArgs: []string{"git", "tag", "--", "v1.0.0", "abc123"},
36 },
37 {
38     //TF-03
39     testName:      "create lightweight tag with force flag",
40     tagName:       "v1.0.0",
41     ref:           "",
42     force:         true,
43     expectedCmdArgs: []string{"git", "tag", "--force", "--", "v1.0.0"},
44 },
45 {
46     //TF-04
47     testName:      "create forced lightweight tag on specific commit",
48     tagName:       "v1.0.0",
49     ref:           "def456",
50     force:         true,
51     expectedCmdArgs: []string{"git", "tag", "--force", "--", "v1.0.0", "def456"},
52 },
53 }
54
55 for _, s := range scenarios {
56     t.Run(s.testName, func(t *testing.T) {
57         runner := oscommands.NewFakeRunner(t)
58         gitCommon := buildGitCommon(commonDeps{runner: runner})
59         tagCommands := NewTagCommands(gitCommon)
60
61         cmdObj := tagCommands.CreateLightweightObj(s.tagName, s.ref, s.force)
62
63         assert.Equal(t, s.expectedCmdArgs, cmdObj.Args())
64     })
65 }
66 }
67
68 func TestTagCommands_CreateAnnotatedObj(t *testing.T) {
69     type scenario struct {
70         testName      string
71         tagName       string
72         ref           string
73         msg           string
74         force         bool
75         expectedCmdArgs []string
76     }
77
78     scenarios := []scenario{
79         {
80             //TF-05
81             testName:      "create annotated tag on HEAD",
82             tagName:       "v1.0.0",

```

```

83     ref:          "",
84     msg:          "Release version 1.0.0",
85     force:        false,
86     expectedCmdArgs: []string{"git", "tag", "v1.0.0", "-m", "Release version
1.0.0"},
87 },
88 {
89     //TF-06
90     testName:     "create annotated tag on specific commit",
91     tagName:      "v1.0.0",
92     ref:          "abc123",
93     msg:          "Major release",
94     force:        false,
95     expectedCmdArgs: []string{"git", "tag", "v1.0.0", "abc123", "-m", "Major
release"},
96 },
97 {
98     //TF-07
99     testName:     "create forced annotated tag",
100    tagName:      "v1.0.0",
101    ref:          "",
102    msg:          "Latest stable",
103    force:        true,
104    expectedCmdArgs: []string{"git", "tag", "v1.0.0", "--force", "-m", "Latest
stable"},
105 },
106 {
107     //TF-08
108     testName:     "create forced annotated tag on specific commit",
109     tagName:      "v1.0.0",
110     ref:          "xyz789",
111     msg:          "Beta version",
112     force:        true,
113     expectedCmdArgs: []string{"git", "tag", "v1.0.0", "--force", "xyz789", "-m",
"Beta version"},
114 },
115 }
116
117 for _, s := range scenarios {
118     t.Run(s.testName, func(t *testing.T) {
119         runner := oscommands.NewFakeRunner(t)
120         gitCommon := buildGitCommon(commonDeps{runner: runner})
121         tagCommands := NewTagCommands(gitCommon)
122
123         cmdObj := tagCommands.CreateAnnotatedObj(s.tagName, s.ref, s.msg, s.force)
124
125         assert.Equal(t, s.expectedCmdArgs, cmdObj.Args())
126     })
127 }
128 }
129

```

```

130 func TestTagCommands_IsTagAnnotated(t *testing.T) {
131     type scenario struct {
132         testName      string
133         tagName         string
134         gitOutput       string
135         gitError        error
136         expectedResult bool
137         expectedError   error
138     }
139
140     scenarios := []scenario{
141     {
142         //TF-09
143         testName:      "tag is annotated",
144         tagName:         "v1.0.0",
145         gitOutput:       "tag\n",
146         gitError:        nil,
147         expectedResult: true,
148         expectedError:   nil,
149     },
150     {
151         //TF-10
152         testName:      "tag is lightweight",
153         tagName:         "v1.0.0",
154         gitOutput:       "commit\n",
155         gitError:        nil,
156         expectedResult: false,
157         expectedError:   nil,
158     },
159     {
160         //TF-11
161         testName:      "tag with extra whitespace",
162         tagName:         "v1.0.0",
163         gitOutput:       " tag \n",
164         gitError:        nil,
165         expectedResult: true,
166         expectedError:   nil,
167     },
168     }
169
170     for _, s := range scenarios {
171         t.Run(s.testName, func(t *testing.T) {
172             runner := oscommands.NewFakeRunner(t).
173                 ExpectGitArgs([]string{"cat-file", "-t", "refs/tags/" + s.tagName},
174                     s.gitOutput, s.gitError)
175
176             gitCommon := buildGitCommon(commonDeps{runner: runner})
177             tagCommands := NewTagCommands(gitCommon)
178
179             result, err := tagCommands.IsTagAnnotated(s.tagName)

```

```

179
180     assert.Equal(t, s.expectedResult, result)
181     if s.expectedError != nil {
182         assert.Error(t, err)
183     } else {
184         assert.NoError(t, err)
185     }
186     runner.CheckForMissingCalls()
187 })
188 }
189 }
190
191 //TF-12
192 func TestTagCommands_LocalDelete(t *testing.T) {
193     runner := oscommands.NewFakeRunner(t).
194         ExpectGitArgs([]string{"tag", "-d", "v1.0.0"}, "", nil)
195
196     gitCommon := buildGitCommon(commonDeps{runner: runner})
197     tagCommands := NewTagCommands(gitCommon)
198
199     err := tagCommands.LocalDelete("v1.0.0")
200
201     assert.NoError(t, err)
202     runner.CheckForMissingCalls()
203 }

```

Abbildung 1: tag_test.go

Use Case	UC1: Release-Version taggen
Akteur	Software-Entwickler
Vorbedingungen	- Repository ist in lazygit geöffnet - Commit für Release ist ausgewählt (z. B. im Commits-View oder Branches-View)
Trigger	Benutzer drückt `n` (new tag)
Hauptscenario	- System zeigt Eingabemaske mit zwei Feldern: - Tag-Name (z. B. "v1.0.0") - Optional: Tag-Beschreibung - Benutzer gibt Tag-Name ein - Benutzer entscheidet: - Beschreibung leer lassen → Lightweight Tag - Beschreibung eingeben → Annotated Tag

	4. System prüft, ob Tag bereits existiert (HasTag) 5. System erstellt Tag auf ausgewähltem Commit 6. System aktualisiert Tags- und Commits-View 7. System zeigt Erfolgsmeldung
Alternativszenarien	4a. Tag existiert bereits: • 4a1. System zeigt Prompt: "Force tag ,v1.0.0'? (Cancel: Esc, Confirm: Enter)" • 4a2. Benutzer bestätigt → Tag wird mit --force überschrieben • 4a3. Benutzer bricht ab → Keine Änderung 5a. GPG-Signierung ist aktiviert: • 5a1. System erstellt immer Annotated Tag (auch ohne Beschreibung) • 5a2. System fordert GPG-Passphrase an • 5a3. Tag wird signiert erstellt 7a. Git-Fehler (z. B. ungültiger Tag-Name): • System zeigt Fehlermeldung • Benutzer kann erneut eingeben
Nachbedingungen	• Tag ist lokal auf dem ausgewählten Commit erstellt • Tag erscheint in der Tags-Liste
Geschäftsregeln	• Annotated Tags werden erstellt bei: Beschreibung vorhanden ODER GPG-Signing aktiviert • Lightweight Tags werden erstellt bei: Keine Beschreibung UND kein GPG-Signing • Force-Flag wird automatisch gesetzt, wenn Tag bereits existiert und Benutzer bestätigt
Häufigkeit	Hoch (bei jedem Release, Milestone, Hotfix)

Use Case	UC2: Fehlerhaften Tag lokal korrigieren
Akteur	Software-Entwickler

Vorbedingungen	• Repository ist geöffnet • Tag existiert lokal • Tag wurde noch nicht gepusht (oder Benutzer ist sich der Konsequenzen bewusst)
Trigger	• Benutzer navigiert zu Tags-View • Wählt fehlerhaften Tag aus • Drückt d (delete)
Hauptszenario	1. System zeigt Menü mit 3 Optionen: • c - Delete local tag • r - Delete remote tag • b - Delete both local and remote 2. Benutzer wählt c (local delete) 3. System führt LocalDelete(tagName) aus 4. System entfernt Tag aus lokaler Datenbank 5. System aktualisiert Tags-View 6. Tag verschwindet aus der Liste
Alternativszenarien	2a. Benutzer wählt Remote Delete: • 2a1. System fragt nach Bestätigung • 2a2. System pusht Tag-Löschung zum Remote 2b. Benutzer wählt Both: • 2b1. Lokaler Tag wird gelöscht • 2b2. Remote Tag wird gelöscht (mit Bestätigung)
Nachbedingungen	• Tag existiert nicht mehr lokal • Benutzer kann neuen Tag mit korrektem Namen/Commit erstellen
Häufigkeit	Mittel (bei Tippfehlern, falschen Commits)

Use Case	UC3: Tag-Informationen anzeigen
Akteur	Software-Entwickler
Vorbedingungen	• Repository ist geöffnet • Tags existieren

Trigger	• Benutzer navigiert zu Tags-View • Wählt einen Tag aus (mit Pfeiltasten)
Hauptszenario	1. System selektiert Tag 2. System ruft <code>IsTagAnnotated(tagName)</code> auf 3. Falls Annotated Tag: • 3a. System ruft <code>ShowAnnotationInfo(tagName)</code> auf • 3b. System zeigt im Main-Panel: ▸ Tagger: Name ▸ TaggerDate: Datum ▸ Tag-Message 4. Falls Lightweight Tag: • 4a. System zeigt Commit-Details (da Tag nur Pointer ist) 5. System zeigt zugehörigen Commit in der Ansicht
Alternativszenarien	Keine relevanten Abweichungen
Nachbedingungen	Benutzer sieht Tag-Details und kann entscheiden (löschen, pushen, checkout)
Häufigkeit	Hoch (bei Code-Review, Release-Vorbereitung)

Use Case	UC4: Bestehenden Tag verschieben (Force-Update)
Akteur	Software-Entwickler
Kontext	"latest"-Tag oder "stable"-Tag soll immer auf aktuellsten Stand zeigen
Vorbedingungen	• Repository ist geöffnet • Tag "latest" existiert auf älterem Commit • Neuer Commit soll getaggt werden
Trigger	Benutzer will Tag aktualisieren
Hauptszenario	1. Benutzer navigiert zu neuem Commit 2. Benutzer drückt n (new tag) 3. Benutzer gibt existierenden Tag-Namen ein (z. B. "latest") 4. System erkennt via <code>HasTag("\latest\")</code>, dass Tag existiert

	5. System zeigt Force-Prompt 6. Benutzer bestätigt 7. System erstellt Tag mit --force Flag 8. Tag wird auf neuen Commit verschoben
Nachbedingungen	• Tag zeigt auf neuen Commit • Alter Commit ist nicht mehr getaggt
Geschäftsregel	• Force-Tags auf Remote können Probleme für andere Entwickler verursachen • Wird oft in CI/CD für Rolling-Tags verwendet
Häufigkeit	Mittel (bei Rolling-Tags, Hotfix-Korrekturen)

Use Case	Führt zu Test
UC1 - Lightweight Tag auf HEAD	CreateLightweightObj - Scenario 1
UC1 - Annotated Tag mit Message	CreateAnnotatedObj - Scenario 1
UC1 - Tag auf älteren Commit	CreateLightweightObj - Scenario 2
UC4 - Tag verschieben (Force)	CreateLightweightObj - Scenario 3 & 4
UC2 - Tag löschen	LocalDelete - Test
UC3 - Tag-Typ erkennen	IsTagAnnotated - Alle Scenarios